

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("Dataset_Day3.csv")

# Compute distance
df['distance'] = np.sqrt(
    (df['dropoff_latitude'] - df['pickup_latitude'])**2 +
    (df['dropoff_longitude'] - df['pickup_longitude'])**2
)

print("Distance for each trip:")
print(df['distance'])

```

```

(myenv) flora_04@Flora:/mnt/d/python/day3$ python3 distance.py
Distance for each trip:
0         0.015140
1         0.022103
2         0.053109
3         0.016528
4         0.051031
...
199995    0.001064
199996    0.022126
199997    0.142223
199998    0.033101
199999    0.048729
Name: distance, Length: 200000, dtype: float64

```

Question 2

```

def detect_outliers(column):
    # Calculate the first and third quartile
    q1 = column.quantile(0.25)
    q3 = column.quantile(0.75)
    # Calculate the Interquartile Range (IQR)
    iqr = q3 - q1
    # Determine the lower and upper bounds for non-outlier values
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    # Return a boolean Series marking True for outliers
    return (column < lower_bound) | (column > upper_bound)

# Identify outliers in specific columns
fare_outlier_mask = detect_outliers(df['fare_amount'])
passenger_outlier_mask = detect_outliers(df['passenger_count'])
distance_outlier_mask = detect_outliers(df['distance'])

# Combine all the outlier conditions
any_outlier_mask = fare_outlier_mask | passenger_outlier_mask | distance_outlier_mask
# Display the indices of rows that are considered outliers
print("Indices of rows with outliers in fare_amount, passenger_count, or distance:")
print(df[any_outlier_mask].index.tolist())
# Create a cleaned dataframe with outliers removed
df_without_outliers = df[~any_outlier_mask]
# Display the shape (rows, columns) of the cleaned dataframe
print("\nShape of cleaned dataset after removing outliers:", df_without_outliers.shape)

```



```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("Dataset_Day3.csv")

# Compute distance
df['distance'] = np.sqrt(
    (df['dropoff_latitude'] - df['pickup_latitude'])**2 +
    (df['dropoff_longitude'] - df['pickup_longitude'])**2
)

plt.figure(figsize=(8, 6))
plt.scatter(df['distance'], df['fare_amount'], alpha=0.5, color='blue')
plt.title('Scatterplot: Distance vs Fare Amount')
plt.xlabel('Distance')
plt.ylabel('Fare Amount')
plt.grid(True)
plt.show()

```

